LATTICE: Ledgered Agent Traces for Transparent, Inspectable Collaborative Execution

Content-Addressed Accountability for Multi-Agent AI Systems

Ali Murtaza Bhutto
alimurtazabhutto@proton.me
ORCID: 0009-0007-2787-943X

Independent Researcher

April 2026

Abstract

Multi-agent AI systems increasingly make consequential decisions, yet lack mechanisms to explain *why* a particular conclusion was reached, what evidence supported it, or whether that evidence has since been invalidated. We present LATTICE, an open-source accountability layer that sits beneath any agent framework and records every agent decision as a content-addressed, cryptographically signed node in a directed acyclic graph (DAG). Each node, called a *Claim*, carries a SHA-256 content hash, an Ed25519 digital signature, a calibrated confidence score, provenance metadata, and references to supporting evidence or upstream claims. LATTICE provides five core capabilities: (1) backward traceability from any conclusion to raw evidence, (2) cryptographic tamper detection through content addressing and signature verification, (3) a revocation waterfall that propagates compromise status through the dependency graph when upstream evidence is invalidated, (4) effective confidence computation that enforces min-propagation across ancestor chains, and (5) zero-friction runtime monitoring that instruments arbitrary Python functions without altering their behavior. We describe the design, architecture, and implementation of LATTICE, evaluate its performance on six benchmark experiments including a synthetic OSINT pipeline case study, and discuss its relationship to formal verification of autonomous systems. The system is framework-agnostic, requires no external services, and is released under the MIT license.

Keywords: multi-agent systems, provenance, accountability, content addressing, cryptographic signatures, directed acyclic graphs, runtime monitoring, formal verification

1 Introduction

The rapid deployment of multi-agent AI systems in domains ranging from open-source intelligence (OSINT) to cybersecurity operations to automated research has created an accountability gap. These systems combine large language model (LLM) reasoning, autonomous planning, tool invocation, memory, and strategic decision-making to produce conclusions that inform real-world actions. Yet the reasoning chains that produce those conclusions are typically opaque: there is no standard mechanism to determine what evidence an agent used, how confident it was, whether its reasoning was sound, or whether the evidence it relied on remains valid.

This accountability gap is not merely an engineering inconvenience. In regulated industries, investigative journalism, intelligence analysis, and security research, an unverifiable claim is worse than no claim at all. When an automated system asserts that a domain is malicious, that a social media account is part of a coordinated campaign, or that a vulnerability exists in critical infrastructure, the consumers of that assertion need to trace it backward to its evidential foundations, verify that no step in the reasoning chain has been tampered with, and determine whether any supporting evidence has since been retracted or invalidated.

Existing approaches to this problem fall into three categories, none of which is fully satisfactory:

1. **Logging.** Systems that record agent actions in flat logs or structured event streams. These pro-

vide a record of what happened but lack the graph structure needed to trace dependencies, the cryptographic guarantees needed to detect tampering, and the uncertainty quantification needed to assess reliability.

2. **Provenance standards.** The W3C PROV data model [1] provides a vocabulary for describing provenance relationships, but it does not prescribe content addressing, cryptographic integrity, or confidence calibration.

3. **Formal verification.** Model checking [2] and runtime verification [3] provide rigorous guarantees about system behavior, but current techniques were developed for systems with well-defined state spaces and deterministic transitions. Adapting them to hybrid symbolic-subsymbolic architectures with stochastic behavior remains an open research problem [4].

LATTICE addresses this gap by providing a lightweight accountability layer that can be inserted beneath any agent framework. It records every agent decision as a *Claim*: a content-addressed, cryptographically signed assertion that references its supporting evidence through a directed acyclic graph.

The contributions of this paper are:

1. A data model for accountable multi-agent execution based on content-addressed, cryptographically signed claim DAGs with calibrated confidence and effective confidence propagation (Section 3).

2. A revocation waterfall mechanism that propagates compromise status through the dependency graph without mutating the immutable historical record (Section 3.5).

3. An implementation providing backward traceability, tamper detection, inflated confidence detection, and zero-friction runtime monitoring with sub-millisecond overhead per instrumented call (Sections 4 and 5).

4. An empirical evaluation including scalability analysis, operation profiling, baseline overhead comparison, and a synthetic OSINT pipeline case study (Section 6).

The system is released as open-source software under the MIT license at https://github.com/thunderstornX/lattice.

2 Related Work

2.1 Provenance and Lineage

Data provenance has a long history in database systems [5] and scientific workflows [6]. The W3C PROV standard [1] provides a general-purpose data model distinguishing entities, activities, and agents.

LATTICE differs from PROV-based systems in three respects: (1) it uses content addressing to make provenance relationships tamper-evident, (2) it integrates cryptographic signatures for agent attribution, and (3) it includes confidence calibration with effective confidence propagation as first-class properties. These additions are motivated by the adversarial threat model inherent in multi-agent AI systems, where agents may produce hallucinated content.

2.2 Content-Addressed Storage

Git [7] uses SHA-1 hashes to identify commits, creating an immutable history. IPFS [8] generalizes this to distributed content-addressed storage using Merkle DAGs. LATTICE applies content addressing to multi-agent decision trails. Each Claim's identifier is the SHA-256 hash of its canonical content, ensuring that any modification produces a different identifier.

2.3 Structured Analytic Techniques

The intelligence community has developed structured analytic techniques (SATs) [9] to mitigate cognitive biases. LATTICE's audit mechanism, which flags unsupported claims, low-confidence assertions, inflated confidence, and broken evidence references, can be seen as an automated implementation of similar principles applied to multi-agent systems.

2.4 Runtime Verification

Runtime verification [3] monitors system behavior during execution. Recent work has addressed multi-agent communication protocol compliance [10, 11] and holistic verification of autonomous systems [4]. The VITAMIN framework [12] provides compositional model checking for multi-agent systems. Ferrando and Malvone [15] introduce rational monitors with game-theoretic reasoning under imperfect information.

LATTICE's runtime monitoring decorator can be understood as a lightweight form of runtime verification: it instruments agent functions to capture provenance relationships as verifiable claims. While LATTICE does not perform formal property checking, the claim DAG it produces provides the raw material for downstream formal analysis.

3 Design

3.1 Design Principles

LATTICE is guided by four principles:

Accountability is infrastructure. Recording provenance, signing claims, and maintaining the dependency graph should happen automatically, without requiring agent developers to modify their code beyond adding a decorator.

Provenance is inseparable from meaning. A claim without documented evidence is an unsupported assertion. LATTICE enforces this by making evidence references a structural property of every claim.

Uncertainty is a first-class object. Every claim carries a confidence score in $[0.0, 1.0]$, with effective confidence propagation ensuring that weak evidence cannot be hidden beneath strong conclusions.

Immutability preserves integrity. Claims, once created, are never modified. Revocation status is stored separately, leaving original claims and their cryptographic signatures intact.

3.2 Core Data Model

Evidence. A content-addressed blob of raw data identified by evidence_id = SHA-256(data).

Claim. A signed assertion referencing its supporting evidence. Formally, a Claim is a tuple:

$$C = (id, a, s, E, \gamma, m, t, \mu, \sigma)$$

where $id = \text{SHA-256}(\text{canonical}(C))$ is the content hash, a is the agent identifier, s is the assertion, $E = \{e_1, \ldots, e_k\}$ is the set of evidence references, $\gamma \in [0, 1]$ is the stated confidence, m is the derivation method, t is the timestamp, μ is metadata, and σ is the Ed25519 signature over id.

Agent. A registered entity with a unique identifier, role label, and Ed25519 keypair.

3.3 The Claim DAG

Claims reference other claims and evidence through their evidence lists, forming a DAG where leaf nodes are raw evidence, intermediate nodes are derived analyses, and root nodes are final conclusions. Cycle detection is enforced at insertion time.

3.4 Effective Confidence

A critical problem with self-reported confidence is that a conclusion can claim high confidence while depending on low-confidence evidence. LATTICE addresses this through *effective confidence*: the minimum confidence across a claim and all of its ancestors.

$$\gamma^*(c) = \min\Big(\gamma(c), \min_{c' \in \text{anc}(c)} \gamma(c')\Big) \tag{1}$$

where $\text{anc}(c)$ denotes the set of all claims reachable by walking backward through the evidence graph from c. Raw evidence leaves are treated as having confidence 1.0.

When $\gamma(c) - \gamma^*(c) > \epsilon$ (with $\epsilon = 0.01$), the claim is flagged as having *inflated confidence*. This flag surfaces during audit, alerting analysts that a conclusion's stated confidence exceeds what its evidence base supports.

This is a conservative, transparent approach: rather than automatically adjusting confidence scores (which would break content addressing and invalidate signatures), LATTICE reports the discrepancy and lets human analysts decide how to act on it.

3.5 Revocation Waterfall

When evidence is discovered to be incorrect, fabricated, or outdated, the corresponding claim can be *revoked*. This is arguably LATTICE's most distinctive contribution: revocation triggers a *waterfall* that propagates through the dependency graph.

Let $\text{deps}(c)$ denote the set of claims that transitively depend on claim c:

$$\text{deps}(c) = \{c' \mid \exists \text{ path } c' \rightsquigarrow c \text{ in the DAG}\}$$

When c is revoked, every $c' \in \text{deps}(c)$ is marked as *compromised*. Crucially, the original claims and their signatures are **never modified**. Revocation status is stored in a separate table, preserving the complete historical record for audit.

This design reflects a fundamental insight: in investigative and intelligence contexts, the fact that

a conclusion was once drawn and later invalidated is itself valuable information. Deleting or mutating compromised claims would destroy this context.

The upstream check (determining whether a given claim depends on any revoked ancestor) uses a recursive CTE walking in the opposite direction, enabling efficient status queries on individual claims.

4 Architecture

LATTICE is organized in four layers:

Storage Layer. SQLite (WAL mode) with four tables: `agents`, `evidence`, `claims`, and `revocations`. The database is a single portable file.

Core Layer. `LatticeStore` provides agent registration, evidence storage, claim creation, DAG traversal, effective confidence computation, audit, verification, and revocation.

Instrumentation Layer. `@lattice_monitor` wraps arbitrary Python functions, automatically capturing inputs, outputs, and timing as signed claims with zero code changes to the wrapped function.

Presentation Layer. CLI (`click` + `rich`) and local web dashboard (FastAPI + inlined D3.js) with force-directed graph visualization, color-coded claim status, and effective confidence display.

5 Implementation

5.1 Content Addressing and Signatures

Claim IDs are computed by serializing content fields to canonical JSON (sorted keys, no whitespace) and hashing with SHA-256. Evidence IDs are the SHA-256 hash of raw data. Each claim is signed by the originating agent's Ed25519 private key over the claim_id.

Verification performs two checks: (1) re-hash the stored content to confirm the claim_id matches (tamper detection), and (2) verify the Ed25519 signature against the agent's public key (attribution verification).

5.2 Revocation Waterfall Implementation

The downstream propagation uses a recursive common table expression (CTE) in SQLite:

```
WITH RECURSIVE downstream(claim_id) AS (
  SELECT c.claim_id
  FROM claims c, json_each(c.evidence) AS je
  WHERE je.value = :revoked_id
  UNION
  SELECT c2.claim_id
  FROM claims c2, json_each(c2.evidence) AS je2
  JOIN downstream d ON je2.value = d.claim_id
)
SELECT claim_id FROM downstream;
```

This leverages SQLite's native graph traversal, avoiding the need to materialize the full DAG in Python and eliminating a class of race conditions.

5.3 Effective Confidence Implementation

The bulk computation uses a recursive memoized traversal in topological order. For each claim, the effective confidence is the minimum of its own confidence and the effective confidences of all its evidence references. Evidence leaves (raw data not in the claims table) are assigned confidence 1.0. This runs in $O(|V| + |E|)$ time.

5.4 Cycle Detection

When a new claim is inserted, LATTICE walks the ancestor chain of each evidence reference using BFS. If the new claim's ID appears upstream, the insertion is rejected with a `CyclicDependencyError`.

5.5 Runtime Monitoring

The `@lattice_monitor` decorator uses `functools.wraps` to transparently instrument functions. It: (1) executes the function, (2) stores the return value as evidence, (3) constructs an assertion from the docstring template, (4) creates a signed claim, and (5) returns the original value unchanged.

6 Evaluation

We evaluate LATTICE across six experiments on a Linux VPS (Intel Xeon E5-2676 v3 @ 2.40GHz, 2GB RAM, SSD, Ubuntu 20.04, Python 3.8). All measurements use `time.perf_counter()` with multiple repetitions.

6.1 Experiment 1: Scalability

We measure the cost of adding one claim to DAGs of increasing size (Table 1). Claim creation includes cycle detection, SQLite insertion, and Ed25519 signing.

Table 1: Claim creation time vs. DAG size

DAG Size	Mean (ms)	Std (ms)
10	0.19	0.07
50	0.43	0.08
100	0.72	0.09
500	3.44	0.33
1,000	7.33	4.18
2,000	12.39	0.28

Claim creation scales approximately linearly with DAG size due to cycle detection traversal. For typical investigations (under 1,000 claims), overhead remains under 10 ms per claim.

6.2 Experiment 2: Operation Profiling

Table 2 profiles individual operations on a 100-claim chain DAG.

Table 2: Operation costs on 100-claim DAG

Operation	Mean (ms)
Evidence storage (dedup)	0.009
Claim creation + signing	0.679
Trace (100-claim chain)	1.359
Effective confidence (single)	1.304
Effective confidence (bulk, 100)	2.435
Audit (100 claims)	4.005
Stats (100 claims)	3.739
Verify all signatures (100+)	43.334

Evidence storage and claim creation are sub-millisecond. Signature verification is the most expensive operation due to Ed25519 public-key operations, but remains under 50 ms for 100 claims. The bulk effective confidence computation (which includes audit integration) adds minimal overhead.

6.3 Experiment 3: Revocation Waterfall

We measure the cost of revoking the root claim in chains of increasing length (Table 3).

Table 3: Revocation waterfall performance

Chain Length	Time (ms)	Claims Affected
10	0.38	10
50	1.02	50
100	3.45	100
500	81.20	500
1,000	350.11	1,000
2,000	1,460.67	2,000

Revocation scales super-linearly because the recursive CTE re-traverses shared subgraphs in diamond-shaped DAGs without memoization: each downstream claim is visited once per path that reaches it, not once per claim. For investigations under 500 claims, waterfall computation completes in under 100 ms. Larger DAGs would benefit from pre-computed dependency indices or incremental materialized views that maintain the transitive closure on each insertion.

6.4 Experiment 4: Effective Confidence Scaling

We measure the cost of computing effective confidence on chains of increasing length (Table 4).

Table 4: Effective confidence computation time

DAG Size	Single (ms)	Bulk (ms)
10	0.15	0.12
50	0.73	0.50
100	1.45	0.88
500	7.55	4.78
1,000	15.81	9.63
2,000	30.20	41.85

The bulk computation is generally faster than individual queries due to memoization. At 2,000 claims, memory pressure causes the bulk method to slow. For practical investigation sizes, both approaches complete well under 50 ms.

6.5 Experiment 5: Instrumentation Overhead

We compare the execution time of a bare Python function against the same function wrapped with `@lattice_monitor`, measured over 1,000 iterations:

- **Bare function:** 0.000235 ms

- **With `@lattice_monitor`:** 0.242 ms
- **Absolute overhead:** 0.242 ms per call

The overhead is dominated by claim creation and SQLite I/O, not the decorator mechanism itself. At 0.24 ms per instrumented call, LATTICE adds negligible latency to any function that performs network I/O, API calls, or LLM inference (which typically take 10–1,000 ms). The percentage overhead is large only relative to the near-zero cost of the bare function itself.

6.6 Experiment 6: Case Study

We evaluate LATTICE on a synthetic OSINT pipeline simulating a three-agent investigation: a *harvester* collecting data from five sources (DNS, WHOIS, HTTP fingerprinting, certificate transparency, passive DNS), an *analyzer* correlating findings into intermediate assessments, and a *reporter* synthesizing a final assessment.

Table 5: OSINT pipeline case study

Metric	**Value**
Agents	3
Claims generated	9
Evidence blobs	5
DAG depth	9
Collection phase	0.807 ms
Analysis phase	0.423 ms
Reporting phase	0.177 ms
Total pipeline time	1.410 ms
All signatures valid	Yes
Inflated confidence flags	1
Final report stated confidence	0.75
Final report effective confidence	0.70
Min effective confidence in DAG	0.70

The pipeline completed in 1.4 ms total, producing 9 signed claims backed by 5 evidence blobs. LATTICE's audit correctly flagged one inflated confidence issue: the final report stated $\gamma = 0.75$ but its effective confidence was $\gamma^* = 0.70$, reflecting the 0.70-confidence attribution claim in the analyzer's chain. This demonstrates that even in a small investigation, effective confidence reveals discrepancies that stated confidence alone would mask.

6.7 Test Coverage

LATTICE includes 89 tests covering: content addressing determinism, Ed25519 signatures, DAG traversal (chains, diamonds), revocation waterfall propagation, cycle detection, effective confidence (single, bulk, diamond DAGs, mixed evidence), inflated confidence audit, CLI output, dashboard API endpoints, and runtime monitor behavior. All tests pass on Python 3.10, 3.11, 3.12, and 3.13.

7 Discussion

7.1 Relationship to Formal Verification

LATTICE is not a formal verification tool. It does not express temporal logic specifications, perform model checking, or provide mathematical guarantees about system behavior. However, it produces the *raw material* that formal verification would operate on: a structured, content-addressed, cryptographically signed record of every agent decision and its dependencies.

We observe four connection points:

Claim DAGs as system traces. The LATTICE DAG is an execution trace of the multi-agent system. Runtime verification techniques [3, 12] could be applied to check temporal properties (e.g., "every conclusion must be supported by at least two independent evidence sources").

Revocation as property violation. The revocation waterfall implements backward reachability analysis. Given a violated property (the revoked claim), LATTICE computes all conclusions that depended on it, structurally similar to counterexample generation in model checking.

Effective confidence as uncertainty representation. Formal verification under imperfect information [13, 14] requires explicit representation of what agents know and do not know. Effective confidence provides a foundation for this representation.

Audit as lightweight specification checking. The audit function checks properties expressible in a specification language: "no claim shall have an empty evidence list" (safety), "all references shall resolve" (well-formedness), "no claim's stated confidence shall exceed its effective confidence by more than ϵ" (consistency).

7.2 Limitations

Single-user model. LATTICE stores private keys in plaintext in SQLite. Multi-user deployments would require encryption at rest and proper key management.

Min-propagation is conservative. Effective confidence uses the minimum across ancestors. More sophisticated approaches (Bayesian propagation, weighted combination) could model confidence degradation more precisely but would require assumptions about independence between evidence sources. We note that min-propagation is actually *correct* under worst-case correlation: if two evidence sources are fully correlated, the joint confidence is bounded by the minimum. In adversarial and security contexts where evidence sources may be manipulated by a common actor, this worst-case posture is a defensible design choice rather than merely a simplification.

LLM as black box. LATTICE captures LLM inputs and outputs but cannot verify internal reasoning. This is a fundamental limitation shared by all monitoring approaches.

Scalability. Cycle detection and revocation waterfall show super-linear scaling beyond 1,000 claims (Tables 1 and 3). Pre-computed dependency indices or incremental algorithms could address this.

7.3 Applications

OSINT investigations. Our case study (Table 5) demonstrates that LATTICE provides accountability infrastructure for the entire investigation lifecycle at negligible performance cost.

Security research. Vulnerability analysis and threat attribution require defensible conclusions. LATTICE's tamper-evident chains and effective confidence provide the audit trail needed for regulatory compliance.

Automated research pipelines. Multi-agent systems conducting literature reviews or data analysis can use LATTICE to document reasoning chains, enabling human reviewers to verify conclusions and identify inflated confidence.

8 Conclusion

We have presented LATTICE, an accountability layer for multi-agent AI systems that records every agent decision as a content-addressed, cryptographically signed claim in a directed acyclic graph. The system provides backward traceability, tamper detection, revocation waterfall propagation, effective confidence computation, and zero-friction runtime monitoring. Empirical evaluation demonstrates sub-millisecond overhead per instrumented call, linear scaling for typical investigation sizes, and effective detection of confidence inflation in a synthetic OSINT pipeline case study.

The source code, benchmarks, and documentation are available at https://github.com/thunderstornX/lattice.

References

[1] L. Moreau and P. Missier, "PROV-DM: The PROV Data Model," W3C Recommendation, April 2013.

[2] E. M. Clarke, O. Grumberg, and D. A. Peled, *Model Checking*. MIT Press, 1999.

[3] M. Leucker and C. Schallhart, "A brief account of runtime verification," *J. Log. Algebraic Methods Program.*, vol. 78, no. 5, pp. 293–303, 2009.

[4] A. Ferrando, L. A. Dennis, R. C. Cardoso, M. Fisher, D. Ancona, and V. Mascardi, "Holistic verification and validation of autonomous systems," *ACM TOSEM*, vol. 30, no. 4, pp. 1–37, 2021.

[5] P. Buneman, S. Khanna, and W. C. Tan, "Why and where: A characterization of data provenance," in *Proc. ICDT*, 2001, pp. 316–330.

[6] J. Freire, D. Koop, E. Santos, and C. T. Silva, "Provenance for computational tasks: A survey," *Computing in Science & Engineering*, vol. 10, no. 3, pp. 11–21, 2008.

[7] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014.

[8] J. Benet, "IPFS – Content Addressed, Versioned, P2P File System," arXiv:1407.3561, 2014.

[9] R. J. Heuer and R. H. Pherson, *Structured Analytic Techniques for Intelligence Analysis*. CQ Press, 2010.

[10] D. Ancona, A. Ferrando, and V. Mascardi, "Parametric runtime verification of multiagent systems," in *Proc. AAMAS*, 2017, pp. 1457–1459.

[11] A. Ferrando, D. Ancona, and V. Mascardi, "Decentralizing MAS monitoring with DecAMon," in *Proc. AAMAS*, 2017, pp. 239–248.

[12] A. Ferrando and V. Malvone, "VITAMIN: A Compositional Framework for Model Checking of Multi-Agent Systems," in *Proc. ICAART*, 2025, pp. 648–655.

[13] F. Belardinelli, A. Lomuscio, and V. Malvone, "An abstraction-based method for verifying strategic properties in multi-agent systems with imperfect information," in *Proc. AAAI*, 2019, pp. 6030–6037.

[14] F. Belardinelli and V. Malvone, "Three-valued strategic abilities," in *Proc. KR*, 2020, pp. 89–98.

[15] A. Ferrando and V. Malvone, "Runtime Verification via Rational Monitor with Imperfect Information," *ACM TOSEM*, vol. 35, no. 3, pp. 74:1–74:29, 2026.

[16] F. Mogavero, A. Murano, G. Perelli, and M. Y. Vardi, "Reasoning about strategies: On the model-checking problem," *ACM Trans. Comput. Log.*, vol. 15, no. 4, pp. 34:1–34:47, 2014.